

Compiladores I

Trabalho Prático 1 — Scanner para a linguagem Micro C

FACOM — Faculdade de Computação
Professor: Amaury Antônio de Castro Junior

Entrega: data a ser definida pelo professor (consulte o AVA/Moodle)

Sumário

1	Visão Geral do Trabalho Prático	1
2	Introdução ao Flex	2
3	Arquivos e Diretórios	4
4	Resultados do Analisador Léxico	5
4.1	Tratamento de Erros	6
4.2	Tabela de Identificadores (String Table)	6
4.3	Strings e Caracteres	7
4.4	Outras Observações	7
5	Notas de Implementação	8
6	Testando o Scanner	8
7	Uso de Ferramentas de Inteligência Artificial	8
7.1	Usos permitidos	9
7.2	Usos não permitidos	9
7.3	Registro do uso de IA	9
7.4	Integridade acadêmica	10
8	O que Entregar	11

1 Visão Geral do Trabalho Prático

Este trabalho prático (e os que o seguirão) irá guiá-lo(a) no projeto e na construção de um compilador para a linguagem **Micro C**. Cada etapa cobrirá uma fase do compilador: análise léxica, análise sintática, análise semântica e geração de código. Ao final, você terá uma fase de compilador funcional, capaz de se conectar às demais fases.

A implementação deverá ser feita em **C**.

Nesta primeira etapa, você deverá escrever um **analisador léxico** (*scanner*) usando o gerador de analisadores léxicos **flex**. Você descreverá o conjunto de tokens de Micro C em um formato de entrada apropriado, e o gerador produzirá o código responsável por reconhecer os tokens em programas Micro C.

A documentação online das ferramentas necessárias ao projeto (manual do flex e demais materiais de apoio) será disponibilizada no AVA da disciplina.

Você poderá realizar este trabalho individualmente ou em dupla, conforme orientação do professor. Não serão aceitos grupos com mais de dois alunos.

2 Introdução ao Flex

O flex permite implementar um analisador léxico escrevendo regras que casam com expressões regulares definidas pelo usuário, executando uma ação especificada para cada padrão reconhecido. O flex compila seu arquivo de regras (por exemplo, `microc.flex`) para um código-fonte em C que implementa um autômato finito reconhecendo as expressões regulares especificadas. Felizmente, não é necessário entender ou mesmo olhar o arquivo gerado automaticamente (geralmente bastante confuso) que implementa suas regras.

Arquivos de regras em flex têm a seguinte estrutura:

```
1 %{  
2 Declaracoes  
3 %}  
4 Definicoes  
5 %%  
6 Regras  
7 %%  
8 Sub-rotinas do usuario
```

As seções *Declarações* e *Sub-rotinas do usuário* são opcionais e permitem escrever declarações e funções auxiliares em C. A seção *Definições* também é opcional, mas é bastante útil, pois permite dar nomes a expressões regulares. Por exemplo, a definição

```
1 DIGIT [0-9]
```

permite definir um dígito. Aqui, `DIGIT` é o nome dado à expressão regular que reconhece qualquer caractere entre 0 e 9. A partir dessa definição, você poderá usar `DIGIT` para definições regulares futuras que utilizem o mesmo padrão pré-definido para dígito (como pode ser observado no código base fornecido para o trabalho). A Tabela 1 apresenta uma visão geral das expressões regulares mais comuns que podem ser especificadas em flex.

Expressão	Significado
<code>x</code>	o caractere “x”
<code>"x"</code>	um “x”, mesmo que x seja um operador
<code>\x</code>	um “x”, mesmo que x seja um operador
<code>[xy]</code>	o caractere x ou y
<code>[x-z]</code>	os caracteres x, y ou z

Expressão	Significado
[^x]	qualquer caractere exceto x
.	qualquer caractere exceto quebra de linha
^x	um x no início de uma linha
<y>x	um x quando o Flex está na condição de início y
x\$	um x no final de uma linha
x?	um x opcional
x*	0, 1, 2, ... ocorrências de x
x+	1, 2, 3, ... ocorrências de x
x y	um x ou um y
(x)	um x
x/y	um x, mas apenas se for seguido de y
{xx}	a tradução de xx a partir da seção de definições
x{m,n}	de m a n ocorrências de x

Tabela 1: Expressões regulares comuns em Flex.

A parte mais importante do seu analisador léxico é a seção de regras. Uma regra em flex especifica uma ação a ser executada caso a entrada corresponda à expressão regular (ou definição prévia) no início da regra. A ação é especificada escrevendo código C comum. Por exemplo, supondo que um dígito represente um token em nossa linguagem (o que **não** é o caso em Micro C, onde números inteiros podem ter múltiplos dígitos), a regra:

```

1 {DIGIT} {
2     microc_yylval.symbol = inttable.add_string(yytext);
3     return DIGIT_TOKEN;
4 }
```

registra o valor do dígito na variável global `microc_yylval` e retorna o código de token apropriado (veja a Seção 5 para uma discussão mais detalhada sobre a variável global `microc_yylval` e a Seção 4.2 para uma discussão sobre a tabela de strings usada no trecho de código acima).

Um ponto importante a lembrar: se a entrada atual corresponder a múltiplas regras, o Flex escolhe a regra que casa com o maior número de caracteres (cadeia mais longa, como visto em sala de aula). Por exemplo, se você definir as duas regras a seguir:

```

1 [0-9]+      { // acao 1 }
2 [0-9a-z]+  { // acao 2 }
```

e se a sequência de caracteres `2a` aparecer em seguida no arquivo sendo escaneado, a ação 2 será executada, pois a segunda regra casa com mais caracteres que a primeira. Se múltiplas regras casarem com o mesmo número de caracteres, a regra que aparece primeiro no arquivo é escolhida.

Ao escrever regras em Flex, pode ser necessário executar ações diferentes dependendo de tokens encontrados anteriormente. Por exemplo, ao processar o token de fechamento de um comentário de bloco, representado pelo padrão “*/” em C, pode ser necessário saber se um comentário de abertura foi encontrado previamente, representado pelo padrão “/*”

em C. Essa estratégia pode ser útil durante a análise sintática, como veremos mais adiante. Uma forma de rastrear estado é declarar variáveis globais na seção de declarações, marcadas como verdadeiras quando certos tokens de interesse são encontrados. Dessa forma, ao encontrar os padrões que definem o token relacionado com a abertura de um comentário, o estado que representa os comentários é ativado e o comentário é todo processado pelas regras associadas ao respectivo padrão. O Flex também oferece uma sintaxe facilitada para esse propósito, por meio de declarações de estado como:

```
1 %Start COMMENT
```

onde **Start** é o nome dado ao estado que pode ser ativado escrevendo **BEGIN(COMMENT)**. Para executar uma ação apenas se um comentário de abertura foi previamente encontrado, você pode condicionar sua regra a **COMMENT** usando a sintaxe:

```
1 <COMMENT> {  
2     // o restante da sua regra ...  
3 }
```

Existe também um estado especial padrão, chamado **INITIAL**, que fica ativo a menos que você indique explicitamente o início de um novo estado. Essa sintaxe pode ser útil para diversos aspectos deste trabalho, como o tratamento de comentários de bloco e o identificação de erros dentro do bloco.

3 Arquivos e Diretórios

Os arquivos-base para este trabalho (esqueleto do analisador léxico, arquivo de teste de exemplo e README) serão disponibilizados pelo professor no **AVA (Moodle)**. Siga as instruções de download e organização de diretórios informadas no AVA.

Os arquivos que você precisará modificar são:

microc.flex Este arquivo contém um esqueleto de descrição léxica para Micro C. Há comentários indicando onde você precisa preencher código, mas isso não é necessariamente um guia completo. Parte do trabalho é garantir que você tenha um analisador léxico correto e funcional. Exceto pelas seções indicadas, você pode modificar livremente o esqueleto fornecido. É possível construir um scanner a partir do esqueleto fornecido, mas ele praticamente não fará nada até que você o complete. Leia o manual do flex para entender as capacidades da descrição fornecida no esqueleto. Quaisquer rotinas auxiliares que você queira escrever devem ser adicionadas diretamente a este arquivo, na seção apropriada (veja os comentários no arquivo).

test.mc Este arquivo contém uma amostra de entrada para ser escaneada. Ele não testa toda a especificação léxica, mas ainda assim é um bom ponto de partida. Não é um bom teste para começar, nem oferece cobertura adequada do seu scanner. Parte do seu trabalho é elaborar boas entradas de teste e uma estratégia de testes. (Não subestime esta etapa — bons casos de teste são difíceis de criar, e esquecer de testar algo é a causa mais comum de perda de pontos na correção.)

Você deve modificar este arquivo com testes que considere adequados para exercitar seu scanner. O **test.mc** fornecido é semelhante a um programa Micro C real, mas

seus testes não precisam ser. Você pode manter quanto quiser do teste fornecido. A cobertura do seu arquivo de teste, ou seja, o quanto ele exercita as diferentes partes do scanner, também será avaliada.

README Este arquivo contém instruções detalhadas sobre o trabalho, bem como dicas úteis. Você deve editá-lo para incluir seu(s) número(s) de matrícula (ver Seção 8 para mais detalhes).

Embora estes arquivos estejam incompletos como fornecidos, o scanner compila e executa. Para compilar o scanner, você deverá seguir as instruções do arquivo `leiname.txt`.

4 Resultados do Analisador Léxico

Neste trabalho, espera-se que você escreva regras em Flex que casem com as expressões regulares que definem os tokens válidos de Micro C, conforme descrito no capítulo de Análise Léxica do livro-texto e na tabela do vocabulário completo (arquivo `tokens.h`), e executem a ação apropriada, como retornar um token do tipo correto, registrar o valor de um lexema quando aplicável, ou reportar um erro quando um erro léxico é encontrado.

Antes de começar, revise a enumeração `TokenType` e a estrutura `Token` descritas no livro-texto. Sua implementação deve definir regras em Flex que casem com as expressões regulares de cada categoria de token e executem a ação correspondente. Por exemplo, ao casar com um token `INTEGERCONST`, seu scanner deve registrar o texto do número (lexema); da mesma forma, ao casar com um `ID`, você precisa registrar o nome do identificador (lexema). Nem todo token exige o armazenamento de informação adicional (atributos); por exemplo, para palavras reservadas como `if`, `else`, `for`, `return`, `int`, `char`, `print` e `main`, basta retornar o tipo do token.

As categorias de tokens de Micro C que seu scanner deve reconhecer são:

- **Tokens fundamentais:** `ID`, `END_OF_FILE`
- **Constantes literais:** `INTEGERCONST`, `CHARCONST`, `STRINGCONST`
- **Operadores aritméticos:** `PLUS (+)`, `MINUS (-)`, `MUL (*)`, `DIV (/)`, `MOD (%)`
- **Operadores relacionais e lógicos:** `EQ (==)`, `NEQ (!=)`, `LT (<)`, `GT (>)`, `LEQ (<=)`, `GEQ (>=)`, `AND (&&)`, `OR (||)`, `NOT (!)`
- **Símbolos de atribuição e pontuação:** `ASSIGN (=)`, `SEMICOLON (;)`, `COMMA (,)`, `LPAREN ( ( ) )`, `RPAREN ( ) )`, `LBRACE ( { )`, `RBRACE ( } )`, `LBRACKET ( [ )`, `RBRACKET ( ] )`
- **Palavras reservadas:** `main`, `if`, `else`, `for`, `return`, `int`, `char`, `print`
- **Token de erro:** `UNDEF` (equivalente ao token `ERROR` do enunciado original)

Seu scanner deve ser robusto, ou seja, deve funcionar para qualquer entrada concebível. Por exemplo, você deve tratar erros como um EOF ocorrendo no meio de uma string ou de um comentário de bloco, bem como caracteres inválidos (aqueles que não pertencem a nenhum padrão da linguagem, como `@` ou `#`). Estes são apenas alguns dos erros que podem ocorrer; consulte o material de apoio da disciplina para os demais.

Você deve garantir o encerramento controlado do programa em caso de erro fatal. *Core dumps* ou exceções não tratadas podem causar perda de pontuação.

4.1 Tratamento de Erros

Todos os erros devem ser encaminhados (futuramente) ao analisador sintático (*parser*). Seu scanner não deve imprimir nada diretamente. Erros são comunicados ao parser retornando um token especial chamado UNDEF (token indefinido). Há vários requisitos para reportar e recuperar-se de erros léxicos:

- Quando um caractere inválido (que não pode iniciar nenhum token) é encontrado, uma string contendo apenas aquele caractere deve ser retornada como o lexema de erro. A leitura deve ser retomada no caractere seguinte.
- Ao reportar o número da linha para uma string ou comentário que se estenda por múltiplas linhas, use a última linha processada. Em geral, o número da linha reportado deve ser aquele em que o token termina. Isso mantém a implementação simples. Caso contrário, seria necessário rastrear estado adicional.
- Se uma string não for fechada antes de uma quebra de linha (`\n`) ou do fim do arquivo (EOF), reporte o erro com a mensagem **"String nao terminada"** e retome a leitura a partir do caractere seguinte à quebra de linha (ou encerre no EOF).
- Se um caractere nulo aparecer dentro de uma string, reporte o erro como **"String contem caractere nulo"**.
- Se um comentário de bloco (`/* ... */`) permanecer aberto quando o EOF for encontrado, reporte esse erro com a mensagem **"EOF em comentario"**. Não gere tokens a partir do conteúdo do comentário simplesmente porque o terminador está ausente.
- Da mesma forma, se um EOF for encontrado antes do fechamento (`"`) de uma string, reporte o erro como **"EOF em string"**.
- Se você encontrar `*/` fora de um comentário de bloco, reporte esse erro como **"Comentario nao iniciado"**, em vez de tokenizá-lo como MUL seguido de RPAREN.
- Lembre-se de que esta fase do compilador captura apenas uma classe muito limitada de erros. **Não** verifique erros que não sejam erros léxicos neste trabalho. Por exemplo, você não deve verificar se variáveis foram declaradas antes do uso. Certifique-se de entender bem quais erros a fase de análise léxica de um compilador verifica e quais ela não verifica antes de começar.

4.2 Tabela de Identificadores (String Table)

Programas costumam ter muitas ocorrências do mesmo lexema. Por exemplo, um identificador geralmente é referenciado mais de uma vez em um programa (senão não seria muito útil!). Para economizar espaço e tempo, uma prática comum em compiladores é armazenar lexemas em uma tabela de strings. Você deve implementar essa tabela usando uma estrutura de dados de sua preferência que deverá ser devidamente comentada e explicada em entrevista com o professor.

Como Micro C não possui classes, tipos definidos pelo usuário, nem identificadores especiais, o scanner deve tratar **todos os identificadores da mesma forma**, independentemente de seu papel posterior no programa (nome de variável, de função ou de parâmetro). Essa distinção caberá às fases posteriores do compilador (análise sintática e semântica).

Não é necessário testar se um literal inteiro cabe na representação especificada para o tipo `int`; basta armazenar o texto completo do literal em seu campo correspondente, independentemente de seu tamanho.

4.3 Strings e Caracteres

O padrão para uma constante de string em Micro C é um caractere de aspas duplas, seguido de zero ou mais caracteres (exceto outra aspas duplas ou uma quebra de linha), seguido de um caractere de aspas duplas de fechamento:

```
1 " [^"\\n]* "
```

O padrão para uma constante de caractere é análogo, delimitado por aspas simples e contendo exatamente um caractere:

```
1 ' [^'\\n] '
```

Seu scanner deve converter sequências de escape¹ em constantes de string e de caractere para seus valores corretos. Por exemplo, `\n` deve ser convertido para o caractere de nova linha, `\t` para tabulação, `\\` para a barra invertida literal, e `\"` para uma aspa dupla literal dentro da string. Assim, se o programador digitar a sequência de caracteres abaixo:

```
1 "a b \n c d"
```

seu scanner deve retornar o token `STRINGCONST` cujo atributo de valor corresponde aos 5 caracteres:

```
1 a b \n c d
```

onde `\n` representa o caractere ASCII literal de nova linha.

4.4 Outras Observações

Seu scanner deve manter a variável `linha_atual`, indicando qual linha do texto-fonte está sendo processada no momento. Esse recurso auxiliará o parser na geração de mensagens de erro úteis.

Comentários de linha única (`//`) e comentários de bloco (`/* */`) devem ser completamente ignorados pelo scanner (não geram tokens), exceto pelos casos de erro descritos na Seção 4.1. Note que, se a especificação léxica estiver incompleta (ou seja, alguma entrada não corresponder a nenhuma expressão regular), os scanners gerados pelo flex apresentam comportamento indesejado. Portanto, antes de entregar o trabalho, certifique-se de que sua especificação seja completa.

¹Uma sequência de escape é uma combinação de caracteres usada em programação para representar comandos especiais, caracteres invisíveis ou símbolos que possuem conflito sintático no código.

5 Notas de Implementação

- Cada chamada ao scanner retorna o próximo token e lexema da entrada. O valor retornado pela função `microc_yylex` é um inteiro representando a categoria sintática (por exemplo, constante inteira, ponto e vírgula, palavra-chave `if`, etc.), conforme definido na enumeração `TokenType` do arquivo `tokens.h` do livro. O lexema é armazenado na variável global `microc_yylval`. Os tokens de símbolos de caractere único (como `;` e `,`) podem ser representados diretamente pelo valor inteiro (ASCII) do próprio caractere, seguindo a convenção adotada na gramática de Micro C.
- Para identificadores, constantes inteiras, constantes de caractere e strings, o lexema deve ser armazenado como uma entrada de símbolo (`Symbol`) na tabela de strings, no campo `microc_yylval.symbol`.
- Uma implementação da tabela de strings pode ser feita, seguindo a mesma lógica descrita no material de apoio do livro-texto: o tipo das entradas da tabela é `Symbol`.
- Quando um erro léxico é encontrado, a rotina `microc_yylex` deve retornar o token `UNDEF`. O lexema é a string representando a mensagem de erro, armazenada no campo `microc_yylval.error_msg` (este campo é uma string comum, não um símbolo). Consulte a Seção 4.1 para as mensagens de erro esperadas.

6 Testando o Scanner

Há pelo menos duas formas de testar seu scanner:

1. **Testes isolados do scanner:** gere entradas de exemplo e execute-as usando o executável `lexer`, que imprime o número da linha e o lexema de cada token reconhecido pelo seu scanner.
2. **Teste integrado:** quando você julgar que seu scanner está funcionando corretamente, você poderá integrá-lo às demais fases do compilador de Micro C (analisador sintático, semântico e gerador de código), à medida que forem desenvolvidas nos trabalhos práticos seguintes, formando um compilador completo que poderá ser testado com qualquer programa de exemplo em Micro C (veja os exemplos do Apêndice C do livro-texto, como Bubble Sort, Torre de Hanói e Fibonacci).

7 Uso de Ferramentas de Inteligência Artificial

O uso de ferramentas de Inteligência Artificial (IA) generativa (por exemplo, ChatGPT, Claude, Gemini, GitHub Copilot, ou qualquer outra ferramenta similar) é **permitido** neste trabalho, desde que sujeito às regras de uso e de registro descritas nesta seção. O objetivo do trabalho é que você aprenda a construir um analisador léxico e compreenda profundamente cada linha do código entregue — a IA pode ser uma aliada nesse processo, mas nunca deve substituí-lo. Independentemente da ferramenta utilizada, **você é o único responsável pelo código entregue** e deverá ser capaz de explicar, justificar e modificar qualquer trecho dele durante a entrevista de avaliação com o professor (ver Seção 4.2).

7.1 Usos permitidos

São considerados usos apropriados de ferramentas de IA, entre outros:

- Esclarecer dúvidas conceituais sobre expressões regulares, a linguagem flex, ou conceitos de análise léxica em geral.
- Ajudar a interpretar mensagens de erro de compilação ou de execução (por exemplo, um erro de sintaxe do flex ou do gcc).
- Revisar um trecho de código *já escrito por você*, sugerindo melhorias de legibilidade, nomes de variáveis, ou identificando bugs pontuais.
- Auxiliar na elaboração de casos de teste adicionais para o arquivo `test.mc`.
- Consultar a sintaxe de comandos, bibliotecas padrão de C, ou a documentação do flex.
- Auxiliar na escrita de comentários e da documentação do código.

Em todos esses casos, trate a IA como trataria um monitor da disciplina ou um colega mais experiente: ela pode orientar, explicar e sugerir caminhos, mas a decisão de projeto e a escrita final do código devem ser suas.

7.2 Usos não permitidos

- Solicitar à IA que gere o arquivo `microc.flex` (ou qualquer outro arquivo do trabalho) completo, ou uma solução substancial para as regras léxicas exigidas, a partir do enunciado ou de trechos dele.
- Copiar e colar, sem compreensão e sem adaptação própria, código gerado por IA para resolver diretamente os itens marcados como `TODO(aluno)` no esqueleto fornecido.
- Utilizar a IA durante a entrevista/defesa oral do trabalho para responder perguntas do professor.
- Utilizar a IA para gerar as mensagens de erro, casos de teste, ou o README de forma a mascarar a ausência de compreensão do trabalho.

O critério prático para diferenciar um uso permitido de um uso não permitido é simples: se você não seria capaz de reproduzir ou explicar o raciocínio por trás de um trecho de código sem consultar a IA novamente, esse uso provavelmente ultrapassou o limite do que é permitido.

7.3 Registro do uso de IA

Caso você utilize qualquer ferramenta de IA generativa durante o desenvolvimento deste trabalho, você deve registrar esse uso em um arquivo de texto chamado `USO_IA.md`, incluído na raiz do pacote entregue junto com `microc.flex`, `test.mc` e `README`. Caso nenhuma ferramenta de IA tenha sido utilizada, inclua o mesmo arquivo apenas com a frase “Nenhuma ferramenta de IA foi utilizada no desenvolvimento deste trabalho.”

Para cada uso relevante de IA, registre uma entrada contendo:

- **Ferramenta e modelo:** nome da ferramenta e, se conhecido, o modelo utilizado (por exemplo, “ChatGPT (GPT-4o)”, “GitHub Copilot”, “Claude”).
- **Trecho ou arquivo afetado:** qual parte do trabalho foi discutida ou modificada com o auxílio da ferramenta (por exemplo, “regra de reconhecimento de comentários de bloco em `microc.flex`”).
- **Finalidade do uso:** um breve resumo do que foi pedido à ferramenta (não é necessário colar a conversa inteira, mas o resumo deve ser específico o suficiente para que o professor entenda o que foi solicitado).
- **O que você fez com a resposta:** se o código sugerido foi usado integralmente, adaptado, ou apenas serviu de inspiração para uma solução escrita por você.

Um exemplo de entrada aceitável:

```
1 ## USO_IA.md
2
3 Ferramenta: ChatGPT (GPT-4o)
4 Trecho: regra de tratamento de comentarios de bloco aninhados
5 Finalidade: pedi uma explicacao de como usar start conditions (%x)
6             do flex para tratar EOF dentro de um comentario aberto
7 O que fiz: entendi a explicacao e escrevi a regra sozinho(a),
8             adaptando o padrao ao formato de erro exigido no
9             enunciado (Secao 4.1)
```

Adicionalmente, inclua no README a linha:

```
1 uso_ia: sim
```

ou

```
1 uso_ia: nao
```

conforme o caso, logo abaixo da(s) linha(s) `aluno:.`

7.4 Integridade acadêmica

O registro do uso de IA **não é penalizado** — pelo contrário, a transparência é valorizada. O que caracteriza violação de integridade acadêmica é:

- Entregar código substancialmente gerado por IA sem o devido registro no arquivo `USO_IA.md`.
- Não ser capaz de explicar, durante a entrevista de avaliação, qualquer trecho do código entregue, tenha ele sido auxiliado por IA ou não.

Caso, durante a correção ou a entrevista, o professor identifique código que o(a) aluno(a) não seja capaz de explicar, esse trecho poderá ser desconsiderado na nota do trabalho, independentemente de estar ou não corretamente registrado no `USO_IA.md`. Casos de uso de IA não declarado, quando identificados, serão tratados como violação de integridade acadêmica, sujeitos às penalidades previstas no regimento da instituição.

8 O que Entregar

Antes de submeter, certifique-se de que você:

- Não deixou nenhuma instrução de depuração (**printf** de debug, por exemplo) no código entregue, e que sua especificação léxica está completa (toda entrada possível corresponde a alguma expressão regular).
- Editou o **README** de forma que a primeira linha inclua seu número de matrícula, no seguinte formato:

```
1 aluno: <sua_matricula>
```

Caso esteja trabalhando em dupla, deve haver **duas** linhas separadas — a submissão será considerada incorreta se ambas as matrículas estiverem na mesma linha. Por exemplo, se dois estudantes com matrículas 2021123456 e 2021654321 trabalham juntos, o **README** deve conter:

```
1 aluno: 2021123456
2 aluno: 2021654321
```

O formato `aluno: 2021123456, 2021654321` **não** será aceito.

- Remova qualquer seção de instruções auxiliares do **README** que porventura tenha sido incluída nos arquivos-base fornecidos, mantendo apenas as informações relevantes para a correção.

A submissão deverá ser feita conforme as instruções específicas informadas pelo professor no **AVA (Moodle)**, por meio do envio de um arquivo compactado (**.zip**) contendo os arquivos do projeto, dentro do prazo estabelecido no cronograma da disciplina. A última versão submetida será a considerada para correção.

Importante: teste sua submissão com antecedência para se certificar de que está no formato correto. É sua responsabilidade verificar eventuais problemas antes do prazo final.

O ônus de convencer o professor de que você compreende o material é seu. Código obscuro e que não seja devidamente explicado terá impacto negativo em sua nota. Portanto, reserve tempo extra para tornar seu código legível e compreensível.